From Words to Answers

How a language model turns your question into a response, and how Retrieval-Augmented Generation (RAG) gives it new knowledge without retraining it — explained simply, with the underlying math alongside.

The Big Picture

There are four layers between what you type and what the model does with it: the **knowledge** a human cares about, translated into **language** (words), which the model converts into **mathematics** (numbers and vectors), which flows through a **neural network** to eventually produce more language back. Everything interesting happens in that middle mathematical layer, invisible to us.

Step 1 — Language Becomes Numbers (Tokenizer)

In plain terms

Your sentence is chopped into small chunks (words or word-pieces), and each chunk is looked up in a giant dictionary that assigns it an ID number. Nothing intelligent has happened yet — it's just indexing, the same way a word gets a page number in a book.

The math

```
"What" → token ID 198 (vocabulary lookup, no math yet)
```

Technical detail

The tokenizer maps each sub-word unit to an integer index in a fixed vocabulary (for example, a vocabulary of 151,936 possible tokens). This step is a pure lookup table operation with no learned computation involved.

Step 2 — Integers Become Vectors (Embeddings)

In plain terms

A single number can't describe what a word means — so instead, each token ID is turned into a long list of numbers (a vector). Imagine coordinates on a map, except instead of just latitude and longitude, there are hundreds of coordinates, each capturing a tiny shade of meaning: how "animal-like", how "large", how "emotional" a concept is. No single number matters on its own — together, they define meaning.

The math

```
Embedding[198] → [-0.32, 0.71, ..., 896 values]
```

Technical detail

An embedding matrix of shape (vocabulary_size × hidden_dimension), e.g. 151,936 × 896, stores one learned row per token. Looking up token ID 198 simply retrieves row 198 of that matrix — an operation, not a computation.

Step 3 — Hidden States (Transformer Layers)

In plain terms

The word's meaning-vector doesn't stay fixed. It passes through many processing stages (commonly 24), and at each stage its meaning is updated based on context — the size of the vector never changes, but what it represents keeps getting refined. For example, "India" starts out just meaning the country, but after reading "capital of India" the vector for India has absorbed the idea that a capital city is relevant nearby.

The math

```
h(0) → Layer_1(h0) → h(1) → ... → Layer_24(h23) → h(24)
```

Technical detail

Each transformer layer is a function that maps a vector of dimension d (e.g. 896) to another vector of the same dimension. Stacking 24 such layers progressively enriches the representation while preserving its shape, allowing contextual information to accumulate across the sequence.

Step 4 — Attention (Deciding What Matters)

In plain terms

This is the core trick. Each word asks: "of everything else in this sentence, what should I pay attention to?" It compares itself to every other word, decides which ones are relevant, and blends in their information. This is how "capital" and "India" end up linked, even though they're separate words.

The math

```
score = Query · Key weights = softmax(score) output = sum(weights × Value)
```

Technical detail

Each token produces a Query, Key, and Value vector. Attention scores are computed as the dot product of a token's Query with every other token's Key. A softmax converts these scores into weights summing to 1, and the final representation is a weighted average of all Value vectors using those weights.

Step 5 — Feed-Forward Network

In plain terms

After gathering context through attention, each word passes through a small processing unit that decides what to do with the information it just received — like a second opinion applied independently to each word.

The math

```
output = Linear2( Activation( Linear1(x) ) )
```

Technical detail

A position-wise multilayer perceptron: a linear transformation, a non-linear activation function, and another linear transformation. It contains the majority of the model's learned parameters.

Step 6 — Final Hidden State

In plain terms

By the time a word's vector has passed through every layer, it no longer just represents that single word — it carries the accumulated context of the whole sentence: grammar, topic, and the specific question being asked.

Technical detail

After the final (24th) layer, the hidden state for the last token encodes the aggregated contextual information needed to predict what comes next, still represented as a single 896-dimensional vector.

Step 7 — LM Head (Scoring Every Possible Word)

In plain terms

The model now needs to guess the next word. It takes its final understanding and checks it against every word it knows, giving each one a raw score for how well it fits.

The math

```
logits = hidden_vector × W_lm_head (896 → 151,936 scores)
```

Technical detail

The final hidden vector (dimension 896) is multiplied by the LM head weight matrix to project it into vocabulary space, producing one raw score (logit) per vocabulary entry — e.g. 151,936 logits.

Step 8 — Softmax (Turning Scores into Probabilities)

In plain terms

Those raw scores aren't probabilities yet — they're just arbitrary numbers, some positive, some negative. A mathematical step called softmax converts them into proper probabilities that all add up to 100%.

The math

```
P(token_i) = e^(logit_i) / sum(e^(logit_j) for all j)
```

Technical detail

Softmax exponentiates each logit and normalizes by the sum of all exponentiated logits, guaranteeing a valid probability distribution over the vocabulary.

Step 9 — Sampling (Picking the Next Word)

In plain terms

With a full list of probabilities, the model must choose one word to output. It can always take the most likely one, or introduce some randomness to make output more varied and natural.

Technical detail

Common strategies: greedy (highest probability), temperature scaling (reshapes the distribution's sharpness), top-k (restrict to the k most likely tokens), and top-p / nucleus sampling (keep the smallest set of tokens whose cumulative probability exceeds a threshold p).

Step 10 — Repeat Until Done

In plain terms

The newly chosen word gets added to the sentence, and the entire process — tokenize, embed, transform, attend, predict — runs again to choose the next word. This continues until the model produces a special "end of sentence" signal.

Technical detail

This is autoregressive generation: each forward pass conditions on all previously generated tokens, and decoding proceeds token-by-token until an end-of-sequence (EOS) token is sampled or a maximum length is reached.

Why This Matters for RAG

The model above is a fixed, frozen probability engine — it only knows what was baked into it during training. It has no idea about your company's HR policy, a private database, or documents written after its training cutoff. Retraining the model every time new information appears would be far too slow and expensive.

Retrieval-Augmented Generation solves this without touching the model at all — it changes only what the model is shown, right before it starts predicting the next token.

Document Embeddings — Reusing the Same Math

In plain terms

Just like individual words get turned into meaning-vectors, entire documents (or chunks of them) get their own meaning-vectors too. A question gets converted the same way. Then the system simply checks which document-vectors sit closest to the question-vector — the closer they are, the more relevant that document probably is.

The math

```
similarity(q, d) = (q · d) / (||q|| × ||d||)
```

Technical detail

Both documents and the incoming query are passed through an embedding model (often a different, smaller model than the generator, producing e.g. 768-dimensional vectors). Cosine similarity between the query vector and each document vector quantifies semantic closeness, independent of vector length.

Retrieval and Prompt Construction

In plain terms

The system pulls out the handful of documents that scored highest on relevance, and simply pastes their content into the prompt, right alongside the original question. The language model then answers using both — the question and the freshly supplied facts.

Technical detail

Top-K nearest document vectors are retrieved from a vector database (e.g. via approximate nearest neighbor search) and concatenated with the user's question into an augmented prompt: [Retrieved Context] + [Question] → SLM → Answer.

The Core Insight

A language model is not a search engine or a database — it is a probability engine that predicts one token at a time. RAG does not teach the model new permanent facts. It feeds the model the right facts immediately before it predicts, every single time it's asked. The model itself never changes; only its input does.

The Full Pipeline at a Glance

Stage	Simple version	Technical version
1. Tokenizer	Words → ID numbers	Vocabulary lookup, integer indices
2. Embedding	IDs → meaning-vectors	Row lookup in (vocab × d) matrix
3. Transformer layers	Meaning gets refined	Stacked functions preserving dim d
4. Attention	Decide what to focus on	Softmax(Q·K) weighted Value average
5. Feed-forward	Process the gathered info	Linear → activation → linear
6. LM head	Score every possible word	hidden × W → logits
7. Softmax	Scores → probabilities	e^x normalized to sum to 1
8. Sampling	Pick the next word	Greedy / temperature / top-k / top-p
9. RAG retrieval	Find relevant facts first	Cosine similarity over doc embeddings

Next phase: instead of studying how the model thinks, the focus shifts to building its memory system — document extraction, chunking, embedding, vector storage, and retrieval — the pipeline that will let this model reason over real enterprise knowledge.